

Action Required on Appspace

Design document

Field	Value
Status	Brought to the product widget standard in v3.0.0 (Space picker, Widget Lab states, empty rule) and harness-tested; v2.0.0 runs on pre1 demo Sites; the live re-test of v3.0.0 and the three-role test are outstanding
Date	5 October 2026
Owner	Edaine Bernadina (Appspace)
Audience	Internal. Nothing here goes to a customer directly; delivery runs through Enhanced Services for scoping and pricing.
Versions	Action Required widget v3.0.0

Contents

1. Problem statement	2
2. Workflow and components	2
2.1 Components	2
2.2 Start-up	3
2.3 Scope "Waiting for me" (pending)	3
2.4 Scope "Pages of chosen Spaces" (space)	3
2.5 Tasks	4
2.6 Rows and opening an item	4
2.7 States, widths and empty cases	4
2.8 The data signals that make it work	5
2.9 Design decisions	5
3. Prerequisites and dependencies	6
3.1 Licensing and platform	6
3.2 Content	6
3.3 Permissions and credentials	6
3.4 Placement surfaces	6
4. Installation	6
4.1 Get the package	6
4.2 Upload and place	7
4.3 Settings	8
4.4 Replacing Must Reads or Waiting for you	9
4.5 Verify	9
5. Using it	9
5.1 Employee	9
5.2 Editor or administrator	10
6. Known limitations	10

7. Deliverables 11

1. Problem statement

Appspace tracks read-and-acknowledge per user: a post, page or story can require an acknowledgment, and each person has a list of what they still owe. The stock place to see that list is the Action Required area of the user profile, which an employee has to go and open. The sources do not name a stock widget that shows the list on a page.

Action Required 1.1.2 showed that pending list on a page the editor controls, with a button per row that opens the post. Two further widgets were then built for the demo Sites around the same data: Must Reads, which lists the tagged pages of one space with Acknowledged or a read time, and Waiting for you, which shows pending items with due dates next to one-off tasks each viewer ticks off. On 5 October 2026 a review of the custom widgets retired both and merged them into Action Required 2.0.0: one acknowledgment widget instead of three, under the same template key. The same day, version 3.0.0 brought it to the product widget standard: a Space picker instead of a typed Space ID, the four Widget Lab states at four sizes, the standard settings groups, translatable wording and the empty-state rule.

The sources do not record a customer behind the widget. The constraints that shaped it are visible in the code: it reads acknowledgment data and links out, it never writes an acknowledgment itself, a placement that only carries the 1.1.2 settings must render exactly as before, and Appspace has no acknowledgment due date, so due dates come from a tag convention.

The problem this design solves: put a person's outstanding reads, acknowledgments and one-off tasks in front of them where they already land, using only the platform's own acknowledgment data and the viewer's own preferences.

2. Workflow and components

2.1 Components

Component	Version	Role
Action Required widget	v3.0.0	Custom widget (React, webpack). Lists what the viewer still has to read and acknowledge, either everything pending or the pages of chosen Spaces, with optional due dates, read time and tasks. Two layouts: Cards or List.
Appspace content feed, read-and-acknowledge	platform	The engine. Decides which posts each user still has to acknowledge, serves the pending list and answers per post whether the viewer has acknowledged it.
Appspace content feed, post listing	platform	Lists the pages of each chosen Space, optionally by tag, for space scope.
User preferences	platform	userpreference/users/me. Holds which tasks the viewer has ticked off.
Console-hosted Widget API	built for 1.13.2	Loaded from the Console at runtime, not bundled. Provides configuration, tenant theme, automatic height, navigation and analytics.

2.2 Start-up

What the widget does when the page opens, in order:

1. Waits for the Widget API (the shell gives up after 10 seconds) and signals loading.
2. Reads its configuration. Missing keys fall back to defaults that behave like 1.1.2. The Wording fields are blank by default and then use the standard wording from the widget's strings file; a filled field honours a language variation that matches the viewer's language key. The strings file is picked by the same language key (English ships).
3. Reads the tenant theme with `getTheme()` and applies it with `applyThemeToDocument()`; with no theme it keeps the Cosmos defaults.
4. In the editor's placeholder mode it draws a data-free wireframe and calls nothing. Otherwise it shows a pulsing skeleton in the shape of the layout.
5. If tasks are configured, reads the viewer's preferences (2.5) in parallel with the content.
6. Loads the content for the chosen scope (2.3 or 2.4), builds one list of content rows and task rows, sorts it, cuts it at How many (never more than 50) and raises `widgetLoaded` with the number of rows shown. `onReady` fires exactly once, on success and on failure.

If the host answers that the schema APIs are not ready yet, the widget retries up to eight times with a growing delay before it shows an error. If the content call fails but tasks are configured, the tasks still show.

2.3 Scope "Waiting for me" (pending)

The default, and the 1.1.2 behaviour.

1. Calls `getMyAcknowledgmentPosts`, which is GET `contentfeed/posts/acknowledgment/me?start={param.start}&limit={param.limit}`, with the viewer's own token, in pages of 50. It stops at the first empty or short page, after 10 pages, or when a later page starts with the same item as the first (the host ignored start).
2. If Spaces are picked, keeps only items whose space or feeds match one of them. The picker's All Spaces value means no filter. If the picker holds only its Current Space token, the widget treats itself as not set up (2.7). If a tag is set, keeps only items carrying it.
3. Keeps the API order, unless due dates are on or tasks exist, in which case rows sort by due date and rows without one go last.

2.4 Scope "Pages of chosen Spaces" (space)

The former Must Reads behaviour.

1. Without a Space the widget calls nothing: editors see "Pick a Space", everyone else sees nothing.
2. For each picked Space, calls `getTaggedPages` (`contentfeed/posts?feedIds={param.spaceId}&postTypes=Page&tags={param.tag}&limit=100`) when a tag is set, otherwise `getSpacePages` (the same without the tag), and merges the pages without duplicates.
3. Keeps published pages only and applies the tag again in the widget. Sorts oldest first. If nothing matches, editors see "Nothing to list here" with the tag, everyone else sees nothing.
4. For each page that has acknowledgment enabled, calls `getPostAcknowledgment` (`contentfeed/posts/{param.postId}?IncludeCurrentUserAcknowledgment=true`) and reads `userAcknowledgment.isAcknowledged`. With Keep acknowledged pages on, the list is cut at How many (50 with Show more on) before these calls. A read that fails counts as not acknowledged and is noted in the diagnostic panel.
5. Acknowledged pages drop out, or, with Keep acknowledged pages on, stay and carry the Acknowledged label.

2.5 Tasks

Up to eight tasks, each with a title, an optional line under it, an optional link and an optional due date (YYYY-MM-DD). Tasks appear in either scope, mixed with the content rows and sorted by due date.

1. A task is identified by a key derived from its title. This is the same key and the same task ids as Waiting for you 1.0.0, so a task ticked off there stays off here.
2. Ticked-off tasks live in the viewer's own preferences: `userpreference/users/me`, under `additionalProperties.waitingForYou.done`, as a map of task key to the time it was ticked off.
3. On Done the widget marks the row Done at once, raises `taskDone`, keeps a copy in the browser's local storage, then re-reads the preferences and PATCHes the whole `additionalProperties` bag back with every other key it found and its own map merged in. If the preferences could not be read at start-up, it writes only the local copy.
4. The row stays visible as Done until the next page load, then it is gone for that viewer.
5. Opening a task raises `taskOpened`. `mailto:` and `tel:` links open directly. Links that start with `#` or `/`, or with the tenant address, open in the same window (a `#` link is prefixed with the Console path). Anything else opens in a new tab.

2.6 Rows and opening an item

The line under a content row is, in this order: the Acknowledged label for an acknowledged page, the due label when due dates are on and the item has a due tag, the read-time label when read time is on. In the List layout, a waiting page with none of these shows the button label as its line.

- Due date: the earliest tag that starts with the due tag prefix and continues with a date, for example `due-2026-10-09`, shown like "Fri 9 Oct". Acknowledged pages get no due date. A task line is its note, or "By" and the date, or empty.
- Read time: the word count of the page body (`captionContent`, otherwise the caption) divided by the reading speed (100 to 400 words a minute), rounded, at least one minute.

The Cards layout draws a thumbnail (cover, banner, thumbnail or first attachment, or a neutral placeholder), the title, the line and one button: the button label for a waiting item, the Acknowledged label as a quieter link for an acknowledged page, or Done and Open for a task. The List layout draws the title as a link with the line under it, and a Done button for tasks.

Opening a content item resolves a link in the same order as 1.1.2:

1. A link carried by the item itself (`acknowledgmentUrl`, `actionUrl`, `url`, `deepLink` or `link`), when the API returns one.
2. The Acknowledge URL template setting, when filled, with `{postId}`, `{origin}` and `{type}` substituted.
3. A Console deep link built from the post type: `{origin}/console/#!/browse/page/{id}`, `.../browse/story/{id}` or `.../browse/post/{id}`. Events route as posts.

`{origin}` is the Console address the host hands the widget, with the page referrer as fallback. Open links in decides the window: Automatic (the default) keeps the earlier behaviour, Cards in a new tab and List in the same window; New tab and Same tab override it. The link goes through the Widget API `navigate()` with a plain new window as fallback. Each open raises `acknowledgmentOpened` with the post id.

2.7 States, widths and empty cases

The widget renders the four Widget Lab states for each layout: Default, Placeholder (static wireframe, no calls), Loading (pulsing skeleton in the layout's shape) and Empty. The layout follows the widget's own width in three bands, narrow below 480 px, medium 480 to 799 and large from 800:

Layout	Narrow (343, 327)	Medium (630)	Large (1024)
Cards	Picture, title and line, the button under them	One row per item, the button in a fixed right column	The same, larger picture, wider column
List	Title over its line, Done on the right	Fixed columns: title, line, Done	The same, wider line column

Case	Editors see	Everyone else sees
Nothing waiting for the viewer	The Nothing waiting text with an illustration	The same (positive empty)
No Space picked in space scope	"Pick a Space" and where to set it	Nothing
Only the picker's Current Space	"Pick a Space": a custom widget cannot tell which Space it is on	Nothing
Only All Spaces, in space scope	"Pick a Space": no one listing covers every Space	Nothing
No matching pages in the Space	"Nothing to list here" with the tag	Nothing
A read refused (401 or 403)	"Hidden for people who cannot read this Space."	Nothing
Any other failure	"Couldn't load. Try again." with the detail and Try again	"Couldn't load. Try again." and Try again

Nothing waiting is a positive empty because for a to-do list it is the answer; a widget that vanished once the viewer was done would look broken. The custom widget host does not tell a widget whether the viewer can edit the page, so on a tenant the editor views appear only with Diagnostic mode on.

2.8 The data signals that make it work

- GET contentfeed/posts/acknowledgment/me returns only what is still pending for the caller, across every space. There is no acknowledged-side list. Verified live on pre1 on 29 September 2026 during work on another widget.
- An empty pending answer therefore means either that everything is acknowledged or that nothing was ever required of this user. Pending scope cannot tell the two apart and shows the empty text for both.
- The space listing does not carry the viewer's acknowledgment. Only the per-post read with IncludeCurrentUserAcknowledgment=true gives the positive signal, so space scope costs one call per page.
- Some listings ignore a filter they document, so the widget applies the tag again itself.
- Only people in a space's publishing target get pending items, and authors never see their own.
- Appspace has no acknowledgment due date, hence the due- tag convention.
- A PATCH of userpreference/users/me replaces the whole additionalProperties bag, hence the re-read right before every write.

2.9 Design decisions

- One widget instead of three. The template key stays widget-action-required-v1, and the defaults equal 1.1.2, so a placement that only carries the three 1.1.2 keys renders exactly as before (checked pixel for pixel in the harness against the 1.1.2 build).
- Read and link, never write an acknowledgment. Acknowledging stays in the post, where the reader sees the content first. The only write is the viewer's own task state.

- Task state keeps the Waiting for you key and storage, so nothing a viewer ticked off is lost in the merge.
- In the Cards layout with Show more off at most four rows are visible; further rows scroll inside the list, as in 1.1.2. With Show more on, How many rows show and a button opens the rest.
- The widget has no title field of its own; the heading comes from the placement. No outer card either: the host owns the frame. Every colour, font and the button radius come from the tenant theme tokens.
- The Space setting became a picker in 3.0.0 under its old name, `spaceId`. The widget reads every shape it has stored: the 3.0.0 list of ids, the 2.0.0 single ID or comma list, and a JSON string, so placements made before the upgrade keep working.

3. Prerequisites and dependencies

3.1 Licensing and platform

- The Employee App with custom widget support and the Console-hosted Widget API. The widget is built for Widget API 1.13.2 and relies on tenant theme inheritance and automatic height.
- The sources record no licence requirement specific to this widget.

3.2 Content

- Pending scope: posts, pages or stories published with read-and-acknowledge to the users who should see them. Without those the list is empty for everyone.
- Space scope: a space with published pages, and, if a tag is used, pages carrying it (for example `must-read`). Pages that should be acknowledged need acknowledgment enabled.
- Due dates: pages tagged with the prefix and a date, for example `due-2026-10-31`.

3.3 Permissions and credentials

- None to set up. Every call runs as the signed-in user: the pending list returns only that user's items, the space listing and per-page reads return what that user may see, and task state goes to that user's own preferences. No passport, no service account and no external service is involved.

3.4 Placement surfaces

- Homepage, community, topic and channel, as declared in the package. A Spaces row also accepts custom widgets. More than one placement per page is allowed. Picker category: News & feeds.
- No other catalog item is required. Must Reads and Waiting for you are retired; this widget replaces them.

4. Installation

4.1 Get the package

In the internal catalog, open Widgets, then Action Required. Download zip gives `widget-action-required-3.0.0.zip`. With an environment connected at the top of the POC tool, Install puts the template straight into that instance instead; placement and configuration remain Console steps either way.

The template identity is the key inside the zip, `widget-action-required-v1`. Note the `-v1` suffix: it is not derivable from the zip name, and an upload under any other key creates a second template.

4.2 Upload and place

1. In Console, open the custom widget templates area and upload `widget-action-required-3.0.0.zip` as a custom widget template. If the template already exists, upload onto it to create the new version.
2. Open the page the widget belongs on. On a homepage, use the theme editor and add the custom widget to a placement. On a Space, add it to a row from the Custom Widget tab. Communities, topics and channels accept it as well.
3. Give the placement a title such as "Action required": the widget draws none of its own.
4. Set the fields below and publish the page. Leave Diagnostic mode off.

A widget that is already placed on a homepage keeps the template version it was placed with (verified on pre1, 29 September 2026). After an upgrade, place the widget again.

4.3 Settings

Setting	Default	What it decides
Content: What to list (scope)	Waiting for me (pending)	Waiting for me: everything the viewer still has to acknowledge, from any Space. Pages of chosen Spaces: the pages of the Spaces below, acknowledged or not.
Content: Spaces or topics (spaceId)	none	Space picker, which also offers Channels and topics. Pages of chosen Spaces: the ones whose pages are listed. Waiting for me: optional filter; none or All Spaces means every Space. A value saved before 3.0.0 (one ID or a comma list) still works.
Content: Only with this tag (tag)	empty	Only items with this tag, for example must-read.
Content: Keep acknowledged pages in the list (showCompleted)	off	Space scope only: acknowledged pages stay and carry the Acknowledged text.
Content: Tasks (tasks)	none	Up to eight rows of title, line under it, link and due date (YYYY-MM-DD). Title and line are translatable.
Display: Layout (layout)	Cards	Cards: picture, title and a button per item. List: a quiet list with a line under each title, Done for tasks.
Display: How many (maxItems)	5	Rows shown, 1 to 50, tasks included.
Display: Show more button (showMore)	off	On: How many rows and a button for the rest, up to 50. Off: four visible at once in Cards, the rest scroll.
Display: Show read time (showReadTime), Reading speed (wordsPerMinute)	off, 200	Read time from the page's word count, 100 to 400 words a minute.
Display: Show due dates (showDueDates), Due date tag prefix (dueTagPrefix)	off, due-	Due date from a due-YYYY-MM-DD tag; items then sort by due date.
Wording: Line at the top (subtitle)	blank	A line above the list.
Wording: Button text (buttonLabel)	blank = Read and acknowledge	The button on each item (Cards), or the line under a waiting page (List).
Wording: Read time text (readLabel)	blank = Read, {minutes} min	{minutes} becomes the read time.
Wording: Due date text (dueLabel)	blank = Read and acknowledge by {date}	{date} is like Fri 9 Oct.
Wording: Acknowledged text (acknowledgedLabel)	blank = Acknowledged	Shown on a page the viewer has acknowledged.
Wording: Nothing waiting text (emptyText)	blank = You have no items that require acknowledgment.	The positive empty line.
Advanced: Open links in (linkTarget)	Automatic	Automatic: Cards open a new tab, List the same window. Or New tab, Same tab.
Advanced: Link address (acknowledgeUrlTemplate)	blank	Overrides the link. Placeholders: {postId}, {origin}, {type}. Blank builds a Console deep link from the post type.

Setting	Default	What it decides
Advanced: Diagnostic mode (debugMode)	off	A technical panel under the widget (settings as read, rows, every API call, raw errors) and the editor views of the empty cases. Also from DevTools with the local storage key ar : debug set to 1. Turn off before publishing.

Wording fields are translatable and blank by default; blank means the standard wording, which is translated with the widget. A placement saved under 2.0.0 keeps the wording it stored.

The Console shows the read-time settings only with Show read time on, the due settings only with Show due dates on, and Keep acknowledged pages only in space scope.

4.4 Replacing Must Reads or Waiting for you

- Must Reads: What to list = Pages of chosen Spaces, the Space and tag it used, Show read time on, Keep acknowledged pages on, Layout = List.
- Waiting for you: What to list = Waiting for me, the Space as a filter (or none for every Space), Show due dates on, the same tasks with the same titles, Layout = List. Tasks ticked off in Waiting for you stay off as long as their titles are unchanged.

These are the configurations the project harness uses for the two old widgets.

4.5 Verify

1. Publish a page or post that requires acknowledgment to a test user. Check the three roles: an editor, a regular employee and someone with no access to the Space.
2. Sign in as that user and open the page carrying the widget. The item appears with its title.
3. Press the button or the title. The post opens (with Open links in on Automatic: a new tab in Cards, the same window in List).
4. Acknowledge it, reload the page with the widget, and confirm the row is gone, or in space scope with Keep acknowledged pages on, that it now carries the Acknowledged label.
5. With tasks configured, press Done on one, reload, and confirm it stays off for that user but still shows for another.
6. If anything is missing, switch on Diagnostic mode on a test page to see the settings as read and every call.

5. Using it

5.1 Employee

The widget lists what still needs a read and an acknowledgment, and any tasks the editor added. In the Cards layout each row has a small image, the title, sometimes a line such as a due date or read time, and a button; in the List layout each row is a title with a line under it (side by side on a wide column). Open an item, read it, and acknowledge it there. Back on the page, the item disappears, or shows as acknowledged, the next time the page loads. A task has a Done button: press it once the task is done and it is off your list for good; Open, where shown, takes you to where the task is done. With nothing outstanding the widget shows one line. If something cannot be loaded it says "Couldn't load. Try again." with a Try again button.

5.2 Editor or administrator

- Place the widget where people land, and give the placement its title.
- For a personal to-do list across the tenant, keep Waiting for me; narrow it to one or more Spaces with the Space picker, or to a tag.
- For a fixed reading list, choose Pages of chosen Spaces, pick the Space and tag the pages; turn on Keep acknowledged pages so people see their progress. If the widget is set up wrongly, ordinary viewers see nothing; turn on Diagnostic mode on a test page to see why.
- To give items a deadline, tag the page with due - and the date and turn on Show due dates. Add tasks for things that are not pages, such as confirming an emergency contact.
- Keep task titles stable: a renamed task counts as a new one and comes back for everyone.
- If employees should land somewhere other than the default Console link, fill the URL template. {type} resolves to page, story or post.
- Usage shows in analytics as widgetLoaded (with the row count), acknowledgmentOpened (with the post id), taskOpened and taskDone (with the task title).

6. Known limitations

- The list loads once per page view. After acknowledging, the row stays until the page is reloaded.
- The widget does not acknowledge anything itself; it only opens the post.
- In pending scope an empty list cannot distinguish "all acknowledged" from "nothing was required" (2.7).
- At most 50 rows, tasks included. With Show more off there is no "more" indicator when the list is cut at How many.
- Pending scope reads at most 10 pages of 50 items before the space and tag filters, and stops early if the host ignores start. Paging past the first 50 items is not recorded as tested on a tenant.
- Space scope lists the first 100 pages of each Space only, with no paging, makes one listing call per picked Space and one acknowledgment call per page that has acknowledgment enabled.
- In space scope a page without acknowledgment enabled is listed as waiting, like an unacknowledged page; the tag is what selects the pages. A page whose acknowledgment cannot be read also counts as not acknowledged.
- Only people in a space's publishing target get pending items, and authors never see their own.
- Appspace has no acknowledgment due date. Due dates depend on editors tagging pages correctly; a mistyped tag simply shows no date. Dates show in the British day format.
- Tasks are identified by their title. Renaming a task brings it back for everyone who ticked it off.
- If the preferences cannot be read, Done is kept only in that browser's local storage.
- The default link uses the Console browse route. The sources do not record a test of that link for an employee who only uses the Employee App; the URL template exists for that case.
- All built-in text is in the strings file, but only English ships. Task titles and lines are marked translatable; what the host passes for a translated sections field is not recorded.
- The custom widget host sends no editor flag, so the editor views of the empty cases (not set up, nothing in the Space, no access) show on a tenant only with Diagnostic mode on. Without it, an editor sees nothing, like everyone else.
- Diagnostic mode shows its panel to every viewer of the placement while it is on.
- The empty illustration is a stand-in for the Cosmos one until design shares it.
- The picker's Current Space cannot be resolved by a custom widget (the host does not pass the Space it sits on), and All Spaces works only as "no filter" in Waiting for me.

- Several Spaces in space scope are read one after another; whether feedIds accepts a list in one call was not relied on.
- Live use is recorded only on pre1 demo Sites; see 7.

7. Deliverables

Package	Notes
widget-action-required-3.0.0.zip	Custom widget template, key widget-action-required-v1. Homepage, community, topic and channel. Twenty-one settings in Content, Display, Wording and Advanced, six API entries, four analytics events. Built by npm run build (webpack and the project's package script) into its dist folder.

Verification status: harness only for v3.0.0, on 5 October 2026; live re-test and the three-role test outstanding. In the harness, headless in Chrome: all 16 matrix cells for both layouts (Cards and List) as an editor and as an employee, the no-access row, the not-set-up and nothing-in-the-Space cases as editor and employee, and scenes for slow, 500, opaque error, 403, no theme, Diagnostic mode, Show more, Open links in, a Done write, and the picker's All Spaces and Current Space values, and the old stored shapes of the Space setting (one ID, a comma list, a JSON string, the new picker list, and 2.0.0 wording stored explicitly): 208 checks, onReady once, no console errors, no sideways scroll. The 15 logic unit tests pass. Earlier history, for v2.0.0: the 2.0.0 logic ported from Must Reads and Waiting for you has unit tests, and the project harness runs 25 scenes on fixtures copied from both old widgets, several of them real API responses; the 1.1.2 configuration was checked pixel for pixel against the 1.1.2 build. On pre1, v2.0.0 is placed on the Safety Hub demo Site in space scope with the tag safety-policy, due dates, the list look and a "Signed off" label, over four acknowledgment pages tagged safety-policy and a due-date, and on other demo Sites. Beyond that placement, live behaviour of v2.0.0 is not recorded: in particular the task preference write, the pending-scope space filter and paging past 50 items. The pending endpoint was verified live on pre1 on 29 September 2026 (pending items only, across every space). Earlier history: the initial version, v1.1.0 for Widget API 1.13.2 compliance, and v1.1.2, which simplified the configuration, all on 8 June 2026. The 2.0.0 zip no longer contains a copy of itself.